Gateway de Visão e Reconhecimento Facial (T800)

Documentação Técnica - Arquitetura, Configuração e Funcionamento

---

Este documento detalha a arquitetura, configuração e funcionamento do nó receptor de vídeo e reconhecimento facial (Gateway). Este módulo é responsável por capturar o fluxo de imagens via rede (ROS2), decodificar os dados e aplicar Inteligência Artificial para reconhecimento biométrico em tempo real, garantindo fluidez extrema através de processamento assíncrono e memória espacial.

1. Ambiente Docker

Para garantir que o sistema rode de forma isolada, sem conflitos de dependências e com acesso à interface gráfica do hospedeiro, toda a aplicação foi conteinerizada utilizando o Docker.

Estrutura de Arquivos Necessária

Para iniciar o ambiente, é necessário criar uma pasta raiz (ex: `DockerGateway`) contendo a seguinte estrutura:

- **Dockerfile:** Arquivo com as instruções de montagem do contêiner.
- **reconhecimento_ros2.py:** O script principal em Python contendo a lógica.
- **cadastrados/:** Pasta contendo as fotos `.jpg` ou `.png` dos rostos que a IA deve reconhecer.

O Dockerfile

O arquivo Dockerfile utiliza a imagem oficial do ROS2 Humble e instala as bibliotecas de visão computacional e IA necessárias:

```
FROM osrf/ros:humble-desktop
RUN apt-get update && apt-get install -y     python3-pip     libgl1-mesa-glx
libglib2.0-0     && rm -rf /var/lib/apt/lists/*
RUN pip3 install --no-cache-dir opencv-python numpy insightface onnxruntime
WORKDIR /app
CMD ["/bin/bash", "-c", "source /opt/ros/humble/setup.bash && python3
reconhecimento_ros2.py"]
```

Como Construir e Executar

No terminal, dentro da pasta raiz, a imagem é construída com o comando:

```
docker build -t t800_vision_node .
```

Para executar, é necessário primeiro liberar a permissão de interface gráfica (`xhost`) e depois subir o contêiner mapeando os volumes e a rede:

```
xhost +local:docker

docker run -it --rm     --network host     -e DISPLAY=$DISPLAY     -e
ROS_DOMAIN_ID=42     -v /tmp/.X11-unix:/tmp/.X11-unix:ro     -v "$(pwd):/app"     -v
"$(pwd)/.insightface:/root/.insightface"     t800_vision_node
```

> **Nota técnica:** O parâmetro `--network host` e a variável `-e ROS_DOMAIN_ID=42` são essenciais para que o contêiner enxergue a mesma camada de rede e o mesmo canal DDS (Data Distribution Service) da máquina emissora.

2. Recepção de Dados e Integração com ROS2

O script foi projetado para atuar como um nó ("Node") passivo no ecossistema ROS2. Ele não requisita a imagem, apenas "escuta" o que está sendo publicado na rede local.

O que é esperado da máquina emissora (Robô/Simulador)

Para que a comunicação ocorra com sucesso, a máquina de origem deve cumprir os seguintes requisitos invisíveis ao nosso código:

- Estar na mesma rede local (LAN/Wi-Fi) sem bloqueios de pacotes UDP.
- Estar operando sob o mesmo `ROS_DOMAIN_ID` (42).
- Publicar as imagens no tópico exato: `/manifold/odin/device0/camera0/compressed`.
- O tipo da mensagem deve ser obrigatoriamente `sensor_msgs/msg/CompressedImage` (formato JPEG), para evitar gargalos de rede.

Configuração do Nó no Código

A recepção dos dados é configurada com um perfil de QoS (Quality of Service) focado em streaming de baixa latência (BEST_EFFORT), priorizando sempre o frame mais recente e descartando pacotes atrasados:

```
class T800VisionNode(Node):
    def __init__(self):
        super().__init__('t800_vision_node')

        qos_profile = QoSProfile(
            reliability=ReliabilityPolicy.BEST_EFFORT,
            history=HistoryPolicy.KEEP_LAST,
            depth=1
        )

        self.subscription = self.create_subscription(
            CompressedImage,
            '/manifold/odin/device0/camera0/compressed',
            self.image_callback,
            qos_profile
        )
```

Tratamento do Dado Recebido

Quando a mensagem chega, o ROS2 aciona o método `image_callback`. O dado bruto (array de bytes comprimido) é extraído via numpy e imediatamente decodificado pelo OpenCV para uma matriz de pixels (matriz BGR) que pode ser exibida na tela e analisada pela IA:

```
    def image_callback(self, msg):
        try:
            np_arr = np.frombuffer(msg.data, np.uint8)
            frame = cv2.imdecode(np_arr, cv2.IMREAD_COLOR)
            if frame is None:
                return
        except Exception:
            return
```

3. Lógica de IA e Reconhecimento Facial

O reconhecimento utiliza a biblioteca InsightFace e foi arquitetado para não congelar o fluxo de vídeo (mantendo os 30 FPS).

Processamento Assíncrono (Threads)

O reconhecimento facial puro consome tempo de CPU. Para evitar travamentos, a inferência da IA é disparada a cada 3 frames em uma Thread separada (em segundo plano). O vídeo continua fluindo na Thread principal enquanto a IA calcula:

```
        # Dispara a IA
        if self.frame_id % RECONHECER_A_CADA == 0 and not self.ia_trabalhando:
            self.ia_trabalhando = True
            threading.Thread(target=self.processar_ia_em_segundo_plano,
args=(frame.copy(),), daemon=True).start()
```

Memória Espacial e Persistência de Identidade

Caso o alvo vire o rosto ou sofra variações de iluminação, a pontuação biométrica pode cair abaixo do limiar (0.60), resultando em "Desconhecido". Para resolver isso, implementamos uma Memória Espacial: o algoritmo calcula a distância matemática entre as coordenadas do rosto "desconhecido" atual e os rostos "conhecidos" do frame anterior. Se a distância for mínima, o sistema herda a identidade anterior, mantendo a consistência visual.

```
            # Lógica de Memória Espacial
            if melhor == "Desconhecido":
                cx_novo = (bbox[0] + bbox[2]) / 2  # Centro X do rosto atual
                cy_novo = (bbox[1] + bbox[3]) / 2  # Centro Y do rosto atual

                # Procura nos rostos que já conhecíamos no frame anterior
                for antigo_box, antigo_nome, antigo_score in self.rostos_atuais:
                    if antigo_nome != "Desconhecido":
                        cx_antigo = (antigo_box[0] + antigo_box[2]) / 2
                        cy_antigo = (antigo_box[1] + antigo_box[3]) / 2

                        distancia = np.sqrt((cx_novo - cx_antigo) ** 2 + (cy_novo -
cy_antigo) ** 2)
                        largura_rosto = antigo_box[2] - antigo_box[0]

                        # Herda a identidade se estiver ocupando o mesmo espaço físico
                        if distancia < largura_rosto:
                            melhor = antigo_nome
                            score = antigo_score
                            break
```

Interpolação Suave (Lerping)

Como a IA atualiza a posição do rosto apenas a cada 3 frames, desenhar o "quadrado verde" diretamente causaria "pulos" visuais. Aplicamos uma interpolação linear (Média ponderada 50/50) nas coordenadas do bounding box, fazendo com que os quadrados deslizem suavemente na tela, acompanhando o rosto com fluidez:

```
                        # Desliza a caixa suavemente (50% do caminho)
                        box_suave = antigo_box * 0.5 + novo_box * 0.5
                        temp_suv.append((box_suave.astype(int), novo_nome,
novo_score))
```

4. Solução de Problemas e Erros Comuns (Linux Mint)

Como o sistema base hospedeiro é o Linux Mint, alguns conflitos de permissões e rede podem ocorrer entre o sistema operacional e o Docker. Abaixo estão os erros mais comuns e suas soluções:

> **4.1. Falha ao abrir a janela de vídeo (Erro de Display)**
> **Erro:** `qt.qpa.xcb: could not connect to display` ou `cannot open display: :0`.
> **Causa:** O sistema de gerenciamento de janelas do Linux Mint (X11) bloqueou o contêiner Docker, impedindo-o de renderizar interfaces gráficas na sua tela por questões de segurança.
> **Solução:** Execute o comando `xhost +local:docker` no terminal do Mint antes de rodar o comando `docker run`.

> **4.2. Permissão negada ao rodar o Docker**
> **Erro:** `permission denied while trying to connect to the Docker daemon socket`.
> **Causa:** Seu usuário no Linux Mint não faz parte do grupo administrativo do Docker.
> **Solução rápida:** Execute o comando do Docker utilizando privilégios de superusuário (`sudo docker run...`).
> **Solução definitiva:** Adicione seu usuário ao grupo do Docker com o comando `sudo usermod -aG docker $USER` e reinicie a máquina.

> **4.3. O script roda sem erros, mas a câmera não aparece**
> **Causa 1 (Firewall do Mint):** O Linux Mint possui um firewall nativo ativo por padrão (ufw). Como o protocolo DDS do ROS2 utiliza broadcast UDP para descobrir outras máquinas, o firewall pode estar bloqueando esses pacotes silenciosamente.
> **Solução:** Verifique o status com `sudo ufw status`. Se estiver ativo, desative temporariamente para o teste utilizando `sudo ufw disable`.
>
> **Causa 2 (Isolamento de Rede AP):** Roteadores corporativos ou universitários costumam ter o "AP Isolation" ativado, o que impede que dois computadores na mesma rede Wi-Fi conversem diretamente.
> **Solução:** Conecte ambas as máquinas via cabo de rede local ou roteie o Wi-Fi através de um smartphone (Hotspot) para contornar o bloqueio do roteador principal.

4.4. Travamento no download inicial do modelo InsightFace

Erro: O terminal congela indefinidamente na mensagem "Carregando modelo InsightFace..." ou exibe um erro de falha de conexão (urllib).

Causa: Na primeira execução, a biblioteca baixa os arquivos matemáticos (modelos .onnx) para a pasta mapeada `.insightface`. Oscilações na rede local podem corromper esse download.

Solução: Cancele o script (Ctrl+C), apague a pasta oculta criada com `rm -rf .insightface` e rode novamente o contêiner conectado a uma rede estável para forçar um novo download limpo.